<SCISE />

v1.0.0 - Production Ready

Sistema de Control de Ingreso y Salida de Equipos

Sustentación de Producto y Proyecto de Software

Problema & Justificación

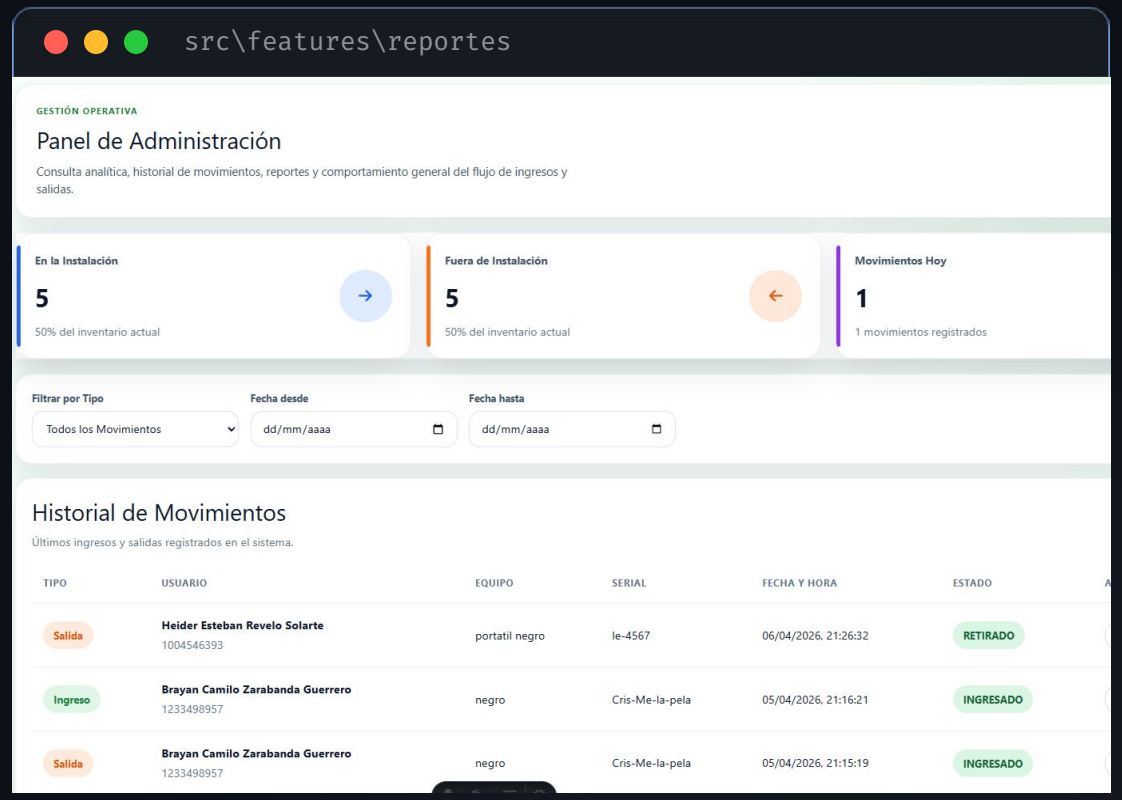
01. Entorno Actual

El registro manual de equipos genera pérdida de trazabilidad, retrasos operativos y dificultad para validar a los responsables.

02. La Solución (SCISE)

Transforma el proceso disperso en un `flujo digital auditable`.

- Centraliza la operación en una plataforma web.
- Garantiza la confiabilidad del dato (quién, qué y cuándo).
- Genera reportes instantáneos para validación CTMA/SENA.



Alcance Funcional

Seguridad & Roles

Autenticación JWT robusta. Separación estricta entre el (acceso total, auditoría) y el (flujo operativo).

`Administrador` `Usuario`

Ingreso / Salida

Identificación ágil mediante escaneo de carnet por cámara web. Registro en lote de activos para agilizar las porterías y laboratorios.

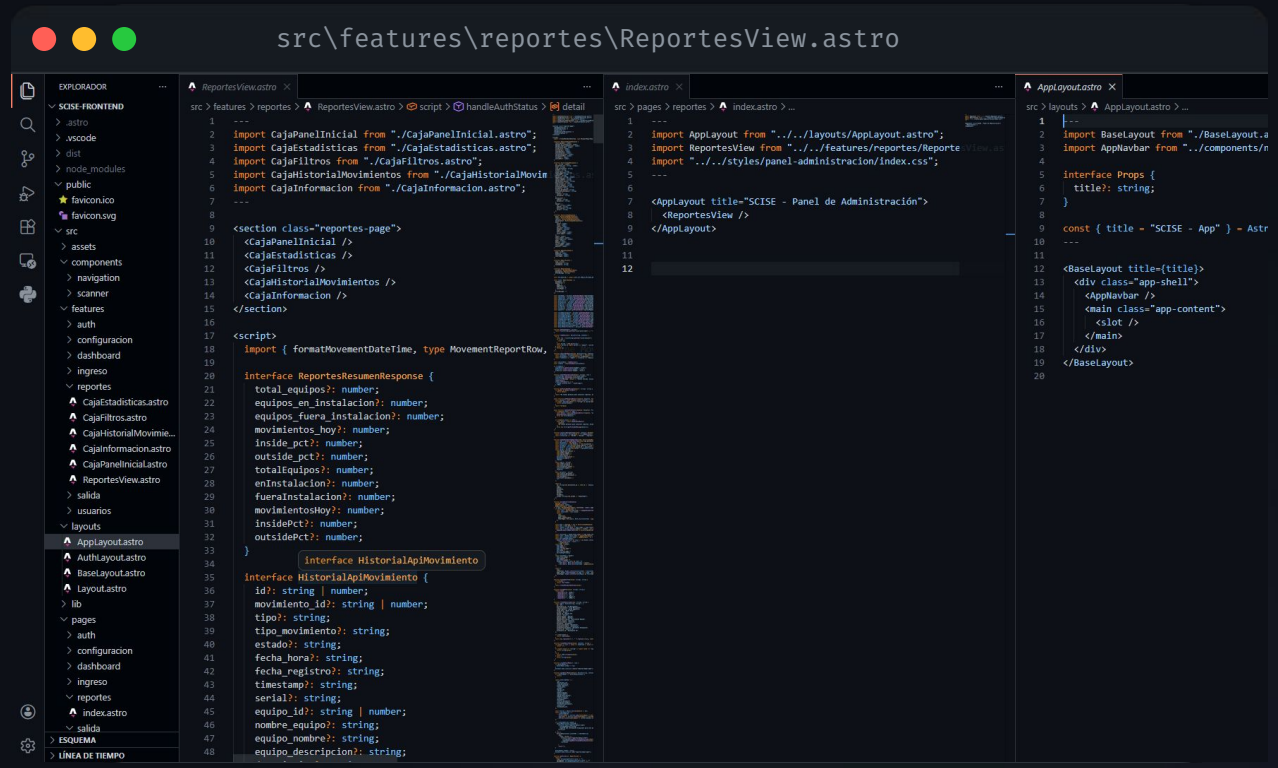
Trazabilidad

Auditoría inmutable de acciones críticas (edición, eliminación, roles). Consultas de historial y exportación de reportes XLSX/PDF.

Arquitectura Frontend

Diseño altamente cohesivo y modularizado para soportar escalabilidad.

- `Astro + TypeScript:` Base del proyecto, tipado estricto y rendimiento.
- `CSS Modular:` Estilos encapsulados por componente.
- `src/features:` Organización por dominios (separación lógica clara).
- `ZXing Integration:` Soporte nativo para lectura de código de barras desde el navegador web.



Arquitectura Backend (Por Capas)

API RESTful robusta que desacopla rutas, lógica y persistencia.

- `api/v1 (Routes):` Exposición de endpoints HTTP (FastAPI).
- `Controllers:` Coordinan entrada/salida y estado HTTP.
- `Services:` Contienen las reglas y lógica de negocio.
- `Repositories:` Acceso a datos abstracto mediante SQLAlchemy.
- `Schemas:` Contratos Request/Response (Pydantic).

app/services/movimientos.py

```python
from fastapi import APIRouter, Depends, status
from sqlalchemy.orm import Session
from app.core.deps import get_current_user, require_role
from app.db.session import get_db
from app.models.usuarios import Usuario
from app.schemas.estudiante import (
    EstudianteCreate,
    EstudianteEstadoUpdate,
    EstudianteLookupResponse,
    EstudianteResponse,
    EstudianteUpdate,
)
from app.schemas.equipo import EquipoAsociadoResponse
from app.controllers import estudiante_controller

router = APIRouter(prefix="/api/v1/estudiantes", tags=["Estudiantes"])


@router.get("", response_model=list[EstudianteResponse])
def listar_estudiantes(
    skip: int = 0,
    limit: int = 200,
    db: Session = Depends(get_db),
    usuario_actual: Usuario = Depends(get_current_user),
):
    return estudiante_controller.listar_estudiantes(skip, limit, db, usuario_actual)


@router.get("/by-documento/{documento}", response_model=EstudianteLookupResponse)
def buscar_estudiante_por_documento(
    documento: str,
    db: Session = Depends(get_db),
    usuario_actual: Usuario = Depends(get_current_user),
):
```

```
(.venv) PS C:\Users\braya\Documents\proyectos\Procesos\SCISE-Project\SCISE-Backend> uvicorn main:app --reload
[2026-04-06 23:12:28] INFO [http] GET /api/v1/dashboard/resumen → 200 (497.7 ms)
INFO:     127.0.0.1:63254 - "GET /api/v1/dashboard/resumen HTTP/1.1" 200 OK
[2026-04-06 23:12:29] INFO [http] GET /api/v1/dashboard/resumen → 200 (481.6 ms)
INFO:     127.0.0.1:61164 - "GET /api/v1/dashboard/resumen HTTP/1.1" 200 OK
[2026-04-06 23:12:29] INFO [http] GET /api/v1/dashboard/historial-reciente → 200 (482.9 ms)
INFO:     127.0.0.1:63254 - "GET /api/v1/dashboard/historial-reciente?page=1&limit=5 HTTP/1.1" 200 OK
[2026-04-06 23:16:46] INFO [http] GET /api/v1/dashboard/resumen → 200 (516.3 ms)
INFO:     127.0.0.1:56627 - "GET /api/v1/dashboard/resumen HTTP/1.1" 200 OK
[2026-04-06 23:16:46] INFO [http] GET /api/v1/dashboard/historial-reciente → 200 (516.6 ms)
INFO:     127.0.0.1:50434 - "GET /api/v1/dashboard/historial-reciente?page=1&limit=5 HTTP/1.1" 200 OK
```

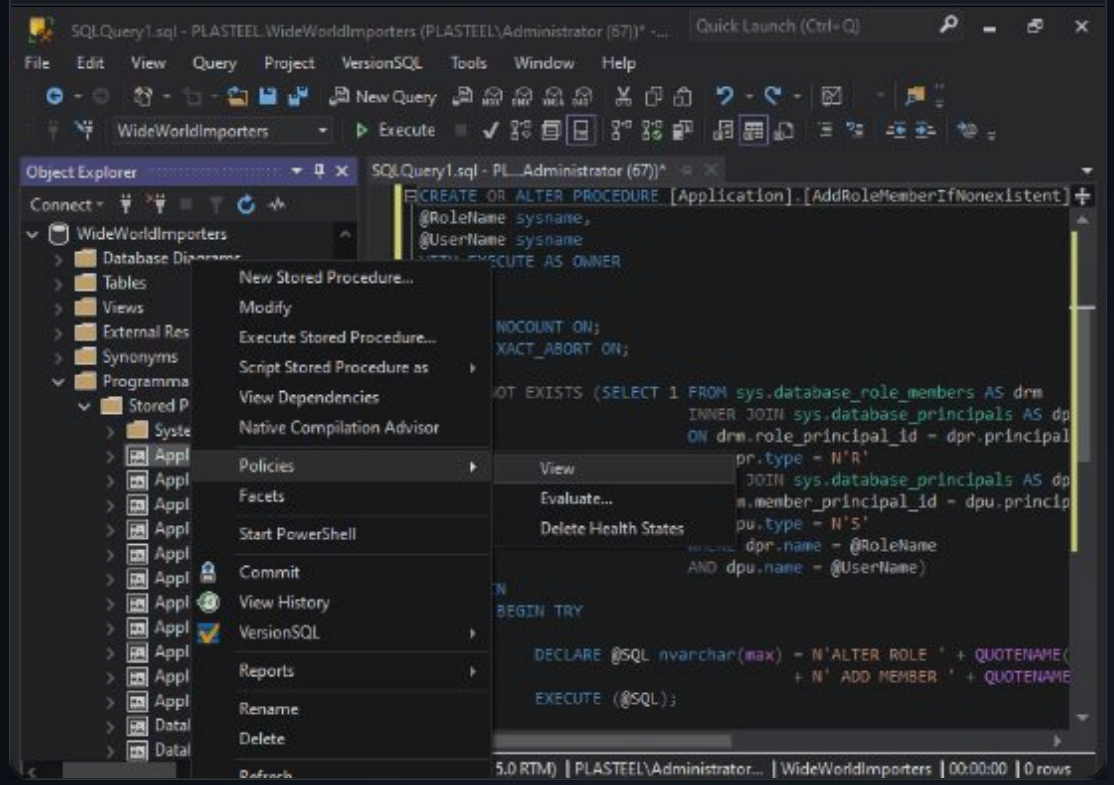
</>Reemplazar con: Código de FastAPI (Ruta o Servicio)

Modelo de Datos (PostgreSQL)

Las relaciones permiten rastrear exactamente qué estudiante posee qué equipos y su historial de tránsito.

- **Usuarios & Roles:** Cuentas de admin/operación y nivel de acceso.
- **Estudiantes:** Portadores identificados por documento/carnet.
- **Equipos:** Activos (nombre, serial, estado).
- **Movimientos:** Tabla transaccional (Ingresos/Salidas + Timestamp).
- **Auditoría:** Log inmutable de acciones del sistema.

models/schema_diagram.sql



Reemplazar con: Diagrama ER o Modelo SQLAlchemy

Integración Frontend-Backend

Hito crítico: Migración de lógica visual hacia el consumo real de la API.

- `API-Only:` Eliminación total de datos mock en módulos operativos.
- `Contratos:` Ajuste estricto de Schemas para asegurar consistencia de nombres, estados y movimientos entre ambas partes.
- `Reflejo Real:` La UI renderiza el estado exacto de la DB, gestionando respuestas asíncronas y manejo de errores.

src/services/api/fetchEquipos.ts

```
message: 'Headers received successfully!',
receivedHeaders: {
  host: 'localhost:3000',
  connection: 'keep-alive',
  'content-type': 'application/json',
  authorization: 'Bearer my-secret-token',
  'custom-header': 'HelloWorld',
  accept: '*/*',
  'accept-language': '*',
  'sec-fetch-mode': 'cors',
  'user-agent': 'node',
  'accept-encoding': 'gzip, deflate',
  'content-length': '24'
```

Retos Técnicos & Soluciones

[Cámara & Escáner] Implementar lectura óptica en navegador.
Solución: Integración de flujo manual/cámara con ZXing y selección dinámica de dispositivo de video.

[Sincronización de Tiempo] Discrepancias de zona horaria entre servidor y cliente.
Solución: Normalización de fechas/horas para Colombia (utilidad compartida en frontend y backend).

[Migración de UI Desacoplada] Sincronizar front con back real sin romper el flujo.
Solución: Refactorización progresiva adoptando consumo API-only módulo por módulo.

[Contratos de Datos] Faltaban campos requeridos por la interfaz en el JSON original.
Solución: Extensión de Pydantic schemas para retornar datos relacionales (ej. nombre del equipo en la salida).

Pruebas y Validación

Se garantizó la estabilidad del sistema mediante validaciones a múltiples niveles:

- `Funcionales:` Flujos completos de Auth, Ingreso (batch), Salida y validación de tiempo de permanencia.
- `Frontend Build:` Astro check para validación estricta de TypeScript.
- `Backend Estructural:` Tests sobre servicios, dependencias y consistencia del ORM.
- `Hardware:` Pruebas del escáner en múltiples navegadores web.

```
npm run dev

AS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS

9 [200] /usuarios 4ms
9 [200] /dashboard 7ms
5 [200] /reportes 3ms
3 [200] /ingreso 4ms
7 [200] /salida 4ms
5 [200] /salida 4ms
7 [200] /usuarios 4ms
5 [200] /ingreso 4ms
5 [200] /salida 3ms
7 [200] /reportes 2ms
3 [200] /dashboard 2ms
5 [200] /reportes 3ms
```

Estado Actual & Impacto

Proyecto en estado funcional avanzado. Reduce errores manuales, aumenta la transparencia del CTMA y centraliza el control institucional.

Preguntas de Sustentación

Gracias por su atención.

```
stack: { Astro, TypeScript, FastAPI, PostgreSQL }
```